```
368             return True
369
370         lat1 = prev["lat_e6"] / 1000000.0
371         lon1 = prev["lon_e6"] / 1000000.0
372         lat2 = q["lat_e6"] / 1000000.0
373         lon2 = q["lon_e6"] / 1000000.0
374         dist_m = _haversine_m(lat1, lon1, lat2, lon2)
375
376         if dist_m >= self.min_distance_m:
377             return True
378         if abs(q["altitude_m"] - prev["altitude_m"]) >= self.min_altitude_change_m:
379             return True
380         if abs(q["heading_de2"] - prev["heading_de2"]) >= self.min_heading_change_de2:
381             return True
382         if abs(q["speed_cms"] - prev["speed_cms"]) >= self.min_speed_change_cms:
383             return True
384         return False
385
386     def ingest_many(self, rows):
387         if not self.enabled or not rows:
388             return
389
390         with self._lock:
391             cur = self._conn.cursor()
392             include_flight_lat_e5 = self._flight_has_lat_e5
393             include_flight_lon_e5 = self._flight_has_lon_e5
394             include_traj_lat_e5 = self._traj_has_lat_e5
395             include_traj_lon_e5 = self._traj_has_lon_e5
396             appended_rows = []
397             for row in rows:
398                 ingest_wallclock = datetime.now(timezone.utc)
399                 icao = str(row.get("icaoAddress", "")).strip().upper()
400                 callsign = str(row.get("callsign", "")).strip().upper()
401                 if not icao and not callsign:
402                     continue
403                 flight_key = icao if icao else callsign
404
405                 ts_iso, ts_dt = _parse_ts(row.get("timestamp"))
406                 q = self._quantize(row)
407                 lat_e5 = int(round(q["lat_e6"] / 10.0))
408                 lon_e5 = int(round(q["lon_e6"] / 10.0))
409                 source_ip = str(row.get("sourceIp", "")).strip()
410
411                 self._sample_counter[flight_key] = self._sample_counter.get(flight_key, 0)
412                 sample_count = self._sample_counter[flight_key]
413
414                 flight_cols = [
415                     "flight_key", "icao_address", "callsign", "timestamp",
416                     "lat_e6", "lon_e6"
417                 ]
418                 flight_vals = [
419                     flight_key, icao or "UNKNOWN", callsign or "UNKNOWN", ts_iso,
420                     q["lat_e6"], q["lon_e6"]
421                 ]
422                 if include_flight_lat_e5:
423                     flight_cols.append("lat_e5")
424                     flight_vals.append(lat_e5)
425                 if include_flight_lon_e5:
426                     flight_cols.append("lon_e5")
427                     flight_vals.append(lon_e5)
428                 flight_cols.extend(["altitude_m", "speed_cms", "heading_de2", "vs_cms", "squawk", "source_ip",
    | "sample_count"])
429                 flight_vals.extend([
430                     q["altitude_m"],
431                     q["speed_cms"],
432                     q["heading_de2"],
433                     q["vs_cms"],
434                     q["squawk"] if q["squawk"] > 0 else None,
435                     source_ip,
436                     sample_count,
437                 ])
438                 update_cols = [c for c in flight_cols if c != "flight_key"]
439                 cur.execute(
```